

1. 구글 (Google)

헤더: Server: gws (Google Web Server)

스타일: "우린 바닥부터 다 직접 만든다." C++로 처음부터 자체 개발한 구글 전용 서버입니다. 전 세계 단위의 트래픽과 구글 내부 인프라에 맞춘 극한의 효율을 자랑합니다.

2. 네이버 (Naver)

헤더: Server: NWS (Naver Web Server)

스타일: "우린 우리 입맛에 맞게 뜯어고친다." 기존 웹 서버(Nginx 등)를 베이스로, 한국형 대규모 동시 접속과 실시간 데이터(뉴스, 검색어 등) 처리에 맞게 극한으로 커스텀했습니다.

```
[
  {
    "id": 1,
    "name": "Leanne Graham",
    "username": "Bret",
    "email": "Sincere@april.biz",
    "address": {
      "street": "Kulas Light",
      "suite": "Apt. 556",
      "city": "Gwenborough",
      "zipcode": "92998-3874",
      "geo": {
        "lat": "-37.3159",
        "lng": "81.1496"
      }
    },
    "phone": "1-770-736-8031 x56442",
    "website": "hildegard.org",
    "company": {
      "name": "Romaguera-Crona",
      "catchPhrase": "Multi-layered client-server neural-net",
      "bs": "harness real-time e-markets"
    }
  },
  {
    "id": 2,
    "name": "Ervin Howell",
    "username": "Antonette",
    "email": "Shanna@melissa.tv",
    "address": {
      "street": "Victor Plains",
      "suite": "Suite 879",
      "city": "Wisokyburgh",
      "zipcode": "90566-7771",
      "geo": {
        "lat": "-43.9509",
        "lng": "-34.4618"
      }
    },
    "phone": "010-692-6593 x09125",
    "website": "anastasia.net",
    "company": {
      "name": "Deckow-Crist",
      "catchPhrase": "Proactive didactic contingency",
      "bs": "synergize scalable supply-chains"
    }
  },
  {
    "id": 3,
    "name": "Clementine Bauch",
    "username": "Samantha",
    "email": "Nathan@yesenia.net",
    "address": {
      "street": "Douglas Extension",
      "suite": "Suite 847",
      "city": "McKenziehaven",
      "zipcode": "59590-4157",
      "geo": {
        "lat": "-68.6102",
        "lng": "-47.0653"
      }
    },
    "phone": "1-463-123-4447",
    "website": "ramiro.info",
    "company": {
      "name": "Romaguera-Jacobson",
```

```

      "catchPhrase": "Face to face bifurcated interface",
      "bs": "e-enable strategic applications"
    }
  },
  {
    "id": 4,
    "name": "Patricia Lebsack",
    "username": "Karianne",
    "email": "Julianne.OConner@kory.org",
    "address": {
      "street": "Hoeger Mall",
      "suite": "Apt. 692",
      "city": "South Elvis",
      "zipcode": "53919-4257",
      "geo": {
        "lat": "29.4572",
        "lng": "-164.2990"
      }
    },
    "phone": "493-170-9623 x156",
    "website": "kale.biz",
    "company": {
      "name": "Robel-Corkery",
      "catchPhrase": "Multi-tiered zero tolerance productivity",
      "bs": "transition cutting-edge web services"
    }
  },
  {
    "id": 5,
    "name": "Chelsey Dietrich",
    "username": "Kamren",
    "email": "Lucio_Hettinger@annie.ca",
    "address": {
      "street": "Skiles Walks",
      "suite": "Suite 351",
      "city": "Roscoeview",
      "zipcode": "33263",
      "geo": {
        "lat": "-31.8129",
        "lng": "62.5342"
      }
    },
    "phone": "(254)954-1289",
    "website": "demarco.info",
    "company": {
      "name": "Keebler LLC",
      "catchPhrase": "User-centric fault-tolerant solution",
      "bs": "revolutionize end-to-end systems"
    }
  },
  {
    "id": 6,
    "name": "Mrs. Dennis Schulist",
    "username": "Leopoldo_Corkery",
    "email": "Karley_Dach@jasper.info",
    "address": {
      "street": "Norberto Crossing",
      "suite": "Apt. 950",
      "city": "South Christy",
      "zipcode": "23505-1337",
      "geo": {
        "lat": "-71.4197",
        "lng": "71.7478"
      }
    },
    "phone": "1-477-935-8478 x6430",
    "website": "ola.org",
    "company": {
      "name": "Considine-Lockman",
      "catchPhrase": "Synchronised bottom-line interface",
      "bs": "e-enable innovative applications"
    }
  },
  {
    "id": 7,
    "name": "Kurtis Weissnat",
    "username": "Elwyn.Skiles",
    "email": "Telly.Hoeger@billy.biz",
    "address": {
      "street": "Rex Trail",
      "suite": "Suite 280",
      "city": "Howemouth",

```

```

      "zipcode": "58804-1099",
      "geo": {
        "lat": "24.8918",
        "lng": "21.8984"
      }
    },
    "phone": "210.067.6132",
    "website": "elvis.io",
    "company": {
      "name": "Johns Group",
      "catchPhrase": "Configurable multimedia task-force",
      "bs": "generate enterprise e-tailers"
    }
  },
  {
    "id": 8,
    "name": "Nicholas Runolfsson",
    "username": "Maxime_Nienow",
    "email": "Sherwood@rosamond.me",
    "address": {
      "street": "Ellsworth Summit",
      "suite": "Suite 729",
      "city": "Aliyaview",
      "zipcode": "45169",
      "geo": {
        "lat": "-14.3990",
        "lng": "-120.7677"
      }
    },
    "phone": "586.493.6943 x140",
    "website": "jacynthe.com",
    "company": {
      "name": "Abernathy Group",
      "catchPhrase": "Implemented secondary concept",
      "bs": "e-enable extensible e-tailers"
    }
  },
  {
    "id": 9,
    "name": "Glenna Reichert",
    "username": "Delphine",
    "email": "Chaim_McDermott@dana.io",
    "address": {
      "street": "Dayna Park",
      "suite": "Suite 449",
      "city": "Bartholomebury",
      "zipcode": "76495-3109",
      "geo": {
        "lat": "24.6463",
        "lng": "-168.8889"
      }
    },
    "phone": "(775)976-6794 x41206",
    "website": "conrad.com",
    "company": {
      "name": "Yost and Sons",
      "catchPhrase": "Switchable contextually-based project",
      "bs": "aggregate real-time technologies"
    }
  },
  {
    "id": 10,
    "name": "Clementina DuBuque",
    "username": "Moriah.Stanton",
    "email": "Rey.Padberg@karina.biz",
    "address": {
      "street": "Kattie Turnpike",
      "suite": "Suite 198",
      "city": "Lebsackbury",
      "zipcode": "31428-2261",
      "geo": {
        "lat": "-38.2386",
        "lng": "57.2232"
      }
    },
    "phone": "024-648-3804",
    "website": "ambrose.net",
    "company": {
      "name": "Hoeger LLC",
      "catchPhrase": "Centralized empowering task-force",
      "bs": "target end-to-end models"
    }
  }
}

```

```

    }
  ]

[{'id': 1,
  'name': 'Leanne Graham',
  'username': 'Bret',
  'email': 'Sincere@april.biz',
  'address': {'street': 'Kulas Light',
  'suite': 'Apt. 556',
  'city': 'Gwenborough',
  'zipcode': '92998-3874',
  'geo': {'lat': '-37.3159', 'lng': '81.1496'}},
  'phone': '1-770-736-8031 x56442',
  'website': 'hildegard.org',
  'company': {'name': 'Romaguera-Crona',
  'catchPhrase': 'Multi-layered client-server neural-net',
  'bs': 'harness real-time e-markets'}},
{'id': 2,
  'name': 'Ervin Howell',
  'username': 'Antonette',
  'email': 'Shanna@melissa.tv',
  'address': {'street': 'Victor Plains',
  'suite': 'Suite 879',
  'city': 'Wisokyburgh',
  'zipcode': '90566-7771',
  'geo': {'lat': '-43.9509', 'lng': '-34.4618'}},
  'phone': '010-692-6593 x09125',
  'website': 'anastasia.net',
  'company': {'name': 'Deckow-Crist',
  'catchPhrase': 'Proactive didactic contingency',
  'bs': 'synergize scalable supply-chains'}},
{'id': 3,
  'name': 'Clementine Bauch',
  'username': 'Samantha',
  'email': 'Nathan@yesenia.net',
  'address': {'street': 'Douglas Extension',
  'suite': 'Suite 847',
  'city': 'McKenziehaven',
  'zipcode': '59590-4157',
  'geo': {'lat': '-68.6102', 'lng': '-47.0653'}},
  'phone': '1-463-123-4447',
  'website': 'ramiro.info',
  'company': {'name': 'Romaguera-Jacobson',
  'catchPhrase': 'Face to face bifurcated interface',
  'bs': 'e-enable strategic applications'}},
{'id': 4,
  'name': 'Patricia Lebsack',
  'username': 'Karianne',
  'email': 'Julianne.OConner@kory.org',
  'address': {'street': 'Hoeger Mall',
  'suite': 'Apt. 692',
  'city': 'South Elvis',
  'zipcode': '53919-4257',
  'geo': {'lat': '29.4572', 'lng': '-164.2990'}},
  'phone': '493-170-9623 x156',
  'website': 'kale.biz',
  'company': {'name': 'Robel-Corkery',
  'catchPhrase': 'Multi-tiered zero tolerance productivity',
  'bs': 'transition cutting-edge web services'}},
{'id': 5,
  'name': 'Chelsey Dietrich',

```

```

import requests
import pandas as pd
from pandas import json_normalize

url = "https://jsonplaceholder.typicode.com/users"

# 1. requests 모듈로 JSON 파일로 불러오기
response = requests.get(url)
json_data = response.json()

# 2. 불러온 데이터를 데이터프레임으로 저장
df = pd.DataFrame(json_data)
print("Original DataFrame head:")
display(df.head())

# 3. 컬럼명(변수명)이 'address'인 series의 내용을 데이터프레임으로 변경
address_df = json_normalize(df['address'])

print("\n'address' column transformed into a new DataFrame head:")
display(address_df.head())

```

Original DataFrame head:

None Like what you see? Visit the [data table notebook](#) to learn more about interactive tables.

'address' column transformed into a new DataFrame head:

	street	suite	city	zipcode	geo.lat	geo.lng
0	Kulas Light	Apt. 556	Gwenborough	92998-3874	-37.3159	81.1496
1	Victor Plains	Suite 879	Wisokyburgh	90566-7771	-43.9509	-34.4618
2	Douglas Extension	Suite 847	McKenziehaven	59590-4157	-68.6102	-47.0653
3	Hoeger Mall	Apt. 692	South Elvis	53919-4257	29.4572	-164.2990
4	Skiles Walks	Suite 351	Roscoeview	33263	-31.8129	62.5342

Error: Runtime no longer has a reference to this dataframe, please re-run this cell and try again.

```
import requests
import pandas as pd
```

```
url_posts = "https://jsonplaceholder.typicode.com/posts"
```

```
# 1. requests 모듈로 JSON 파일 불러오기
response_posts = requests.get(url_posts)
json_data_posts = response_posts.json()
```

```
# 2. 불러온 데이터를 데이터프레임으로 저장
df_posts = pd.DataFrame(json_data_posts)
print("Posts DataFrame head:")
display(df_posts.head())
```

Posts DataFrame head:

	userId	id	title	body
0	1	1	sunt aut facere repellat provident occaecati e...	quia et suscipit\nsuscipit recusandae consequu...
1	1	2	qui est esse	est rerum tempore vitae\nsequi sint nihil repr...
2	1	3	ea molestias quasi exercitationem repellat qui...	et iusto sed quo iure\nvoluptatem occaecati om...
3	1	4	eum et est occaecati	ullam et saepe reiciendis voluptatem adipisci\...
4	1	5	nesciunt quas odio	repudiandae veniam quaerat sunt sed\nalias aut...

```
# 3. index가 짝수인 내용만 필터링
df_even_indexed = df_posts[df_posts.index % 2 == 0]
```

```
print("Even indexed posts DataFrame head:")
display(df_even_indexed.head())
```

Even indexed posts DataFrame head:

	userId	id	title	body
0	1	1	sunt aut facere repellat provident occaecati e...	quia et suscipit\nsuscipit recusandae consequu...
2	1	3	ea molestias quasi exercitationem repellat qui...	et iusto sed quo iure\nvoluptatem occaecati om...
4	1	5	nesciunt quas odio	repudiandae veniam quaerat sunt sed\nalias aut...
6	1	7	magnam facilis autem	dolore placeat quibusdam ea quo vitae\nmagni q...
8	1	9	nesciunt iure omnis dolorem tempora et accusan...	consectetur animi nesciunt iure dolore\nenim q...

```
output_filename = 'even_indexed_posts.xlsx'
df_even_indexed.to_excel(output_filename, index=False)
```

```
print(f"Data saved to {output_filename}")
```

```
# Google Colab에서 파일 다운로드 링크 생성
from google.colab import files
files.download(output_filename)
```

Data saved to even_indexed_posts.xlsx

```
import requests
import pandas as pd
import gradio as gr
```

```
# url = "https://jsonplaceholder.typicode.com/posts" 에서 데이터를 불러와
# requests.get(url, params=params)를 사용하여 사용자가 Gradio UI를 통해 userId를
# 입력하면 실시간으로 해당 데이터를 필터링하여 Pandas 데이터프레임으로 보여주는
# 코드를 작성하라
```

```
def get_posts_by_userid(user_id):
    url = "https://jsonplaceholder.typicode.com/posts"
    params = {'userId': user_id}
    response = requests.get(url, params=params)
    json_data = response.json()
    df = pd.DataFrame(json_data)
    return df

# Gradio 인터페이스 생성
iface = gr.Interface(
    fn=get_posts_by_userid,
    inputs=gr.Number(label="Enter User ID (1-10)", precision=0),
    outputs=gr.DataFrame(label="Filtered Posts"),
    title="Filter Posts by User ID",
    description="Enter a User ID to fetch posts associated with that user from JSONPlaceholder."
)

# 인터페이스 실행
iface.launch()
```

It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically setting `share=True` (you

Colab notebook detected. To show errors in colab notebook, set debug=True in launch())

* Running on public URL: <https://61f71e55c218047430.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working dir

Filter Posts by User ID

Enter a User ID to fetch posts associated with that user from JSONPlaceholder.

Enter User ID (1-10)

5

Clear
Submit

Filtered Posts

user...	id	title
5	41	non est facere
5	42	commodi ullam sint et excepturi error
5	43	eligendi iste nostrum consequuntur ac
5	44	optio dolor molestias sit
5	45	ut numquam possimus omnis eius suscip
5	46	aut quo modi neque nostrum ducimus
5	47	quibusdam cumque rem aut deserunt
5	48	ut voluptatem illum ea doloribus itac

다음 단계:  Cloud Run에 배포

```
import pandas as pd
import os

# 파일명 목록
file_names = [
    '서울특별시 공공자전거 이용정보(일별)_2510.csv',
    '서울특별시 공공자전거 이용정보(일별)_2511.csv',
    '서울특별시 공공자전거 이용정보(일별)_2512.csv'
]

# 모든 데이터프레임을 저장할 리스트
dfs = []

# 각 파일을 읽어와 리스트에 추가
for file_name in file_names:
    try:
        # Colab 환경에 업로드된 파일을 읽기 위해 경로 설정
        file_path = os.path.join('/content/', file_name)
        df = pd.read_csv(file_path, encoding='cp949') # 한글 인코딩 고려
        dfs.append(df)
        print(f"Successfully loaded {file_name}")
    except FileNotFoundError:
        print(f"Error: {file_name} not found. Please ensure the file is uploaded to /content/.")
    except Exception as e:
```

```
print(f"Error loading {file_name}: {e}")

# 데이터프레임 병합
if dfs:
    merged_df = pd.concat(dfs, ignore_index=True)
    print("\nMerged DataFrame head:")
    display(merged_df.head())
    print("\nMerged DataFrame info:")
    merged_df.info()
else:
    print("No dataframes were loaded. Please check file names and paths.")
```

Successfully loaded 서울특별시 공공자전거 이용정보(일별)_2510.csv
 Successfully loaded 서울특별시 공공자전거 이용정보(일별)_2511.csv
 Successfully loaded 서울특별시 공공자전거 이용정보(일별)_2512.csv

Merged DataFrame head:

	대여일자	대여소번호	대여소	대여구분코드	성별	연령대	이용건수	운동량	탄소량	이동거리(M)	이용시간(분)
0	2025-10-01	739	739. 신월사거리	정기권	NaN	~10대	1	58.63	0.51	2209.88	15
1	2025-10-01	1024	1024. 강동구청 앞	정기권	NaN	~10대	1	46.08	0.42	1790.06	15
2	2025-10-01	1964	1964. 원메디타운 앞	정기권	NaN	~10대	1	18.89	0.28	1192.45	9
3	2025-10-01	1966	1966. 한마을아파트 정문상가	정기권	NaN	~10대	1	14.26	0.21	899.94	4
4	2025-10-01	1160	1160. 양천향교역 7번출구앞	정기권	NaN	~10대	1	12.89	0.14	591.77	3

Merged DataFrame info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5002107 entries, 0 to 5002106
Data columns (total 11 columns):
#   Column  Dtype
---  -
0   대여일자    object
1   대여소번호  int64
2   대여소      object
3   대여구분코드 object
4   성별        object
5   연령대      object
6   이용건수    int64
7   운동량      float64
8   탄소량      float64
9   이동거리(M) float64
10  이용시간(분) int64
dtypes: float64(3), int64(3), object(5)
memory usage: 419.8+ MB
```

2. 기본 통계 분석 및 Gradio UI 생성

병합된 데이터를 바탕으로 기본 통계 분석을 수행하고, 여러 화면을 가진 Gradio UI를 생성합니다. UI는 데이터 요약, 기본 통계, 그리고 몇 가지 시각화를 포함합니다.

```
import gradio as gr
import matplotlib.pyplot as plt
import seaborn as sns
import io
import pandas as pd
import os # Import os for path checking

# --- Font Installation and Configuration ---
# Check if font is already installed. If not, install it.
font_path = '/usr/share/fonts/truetype/nanum/NanumBarunGothic.ttf'
if not os.path.exists(font_path):
    print("Korean font 'NanumBarunGothic' not found. Installing now...")
    !sudo apt-get update -qq > /dev/null 2>&1
    !sudo apt-get install -y fonts-nanum > /dev/null 2>&1
    !sudo fc-cache -fv > /dev/null 2>&1 # Rebuild font cache
    print("Nanum font installed. PLEASE RESTART RUNTIME (Runtime -> Restart runtime...) AND RE-RUN ALL CELLS for changes to take effect")

# Configure matplotlib for Korean characters (this part should run AFTER font is available)
try:
    import matplotlib.font_manager as fm
    # Re-add font to font manager if it exists and after restart
    if os.path.exists(font_path):
        font_prop = fm.FontProperties(fname=font_path)
        font_manager.fonts.append(font_prop)
```

```

        fm.fontManager.addfont(font_path)
    plt.rcParams['font.family'] = 'NanumBarunGothic'
    plt.rcParams['axes.unicode_minus'] = False # Solve the minus sign problem in Korean font
    print("Matplotlib configured to use NanumBarunGothic.")
except Exception as e:
    print(f"Error configuring Matplotlib with NanumBarunGothic: {e}. Falling back to default.")

def get_dataframe_head(df):
    if df is None or df.empty:
        return "DataFrame is empty or not loaded."
    return df.head().to_html()

def get_dataframe_info(df):
    if df is None or df.empty:
        return "DataFrame is empty or not loaded."
    info_buffer = io.StringIO()
    df.info(buf=info_buffer)
    return info_buffer.getvalue()

def get_descriptive_statistics(df):
    if df is None or df.empty:
        return "DataFrame is empty or not loaded."
    return df.describe().to_html()

def plot_daily_usage(df):
    if df is None or df.empty:
        return None

    # '대여일자' 컬럼을 datetime 형식으로 변환
    if '대여일자' in df.columns:
        # Ensure '대여일자' is in datetime format for plotting
        df_copy = df.copy() # Work on a copy to avoid SettingWithCopyWarning
        df_copy['대여일자'] = pd.to_datetime(df_copy['대여일자'], errors='coerce')
        df_copy.dropna(subset=['대여일자'], inplace=True)
        daily_usage = df_copy.groupby(df_copy['대여일자'].dt.date).size().reset_index(name='count')
        daily_usage['대여일자'] = pd.to_datetime(daily_usage['대여일자'])

        plt.figure(figsize=(12, 6))
        sns.lineplot(x='대여일자', y='count', data=daily_usage)
        plt.title('일별 공공자전거 이용 횟수')
        plt.xlabel('날짜')
        plt.ylabel('이용 횟수')
        plt.xticks(rotation=45)
        plt.grid(True)
        plt.tight_layout()

        plot_path = "daily_usage_plot.png"
        plt.savefig(plot_path)
        plt.close()
        return plot_path
    else:
        return "'대여일자' column not found for plotting."

def plot_gender_distribution(df):
    if df is None or df.empty:
        return None

    if '성별' in df.columns:
        gender_counts = df['성별'].value_counts(dropna=False)

        plt.figure(figsize=(8, 6))
        sns.barplot(x=gender_counts.index, y=gender_counts.values, palette='viridis')
        plt.title('이용자 성별 분포')
        plt.xlabel('성별')
        plt.ylabel('이용 횟수')
        plt.tight_layout()

        plot_path = "gender_distribution_plot.png"
        plt.savefig(plot_path)
        plt.close()
        return plot_path
    else:
        return "'성별' column not found for plotting."

def plot_hourly_usage(df):
    if df is None or df.empty:
        return None

    if '대여일자' in df.columns:
        df_copy = df.copy()
        df_copy['대여일자'] = pd.to_datetime(df_copy['대여일자'], errors='coerce')
        df_copy.dropna(subset=['대여일자'], inplace=True)
        df_copy['hour'] = df_copy['대여일자'].dt.hour

```



```

hourly_usage = df_copy.groupby('hour').size().reset_index(name='count')

plt.figure(figsize=(10, 6))
sns.barplot(x='hour', y='count', data=hourly_usage, palette='magma')
plt.title('시간대별 공공자전거 이용 현황')
plt.xlabel('시간 (시)')
plt.ylabel('이용 횟수')
plt.tight_layout()

plot_path = "hourly_usage_plot.png"
plt.savefig(plot_path)
plt.close()
return plot_path
else:
    return "'대여일자' column not found for plotting."

def plot_age_group_distribution(df):
    if df is None or df.empty:
        return None
    if '연령대' in df.columns:
        age_group_counts = df['연령대'].value_counts(dropna=False).sort_index()

        plt.figure(figsize=(10, 6))
        sns.barplot(x=age_group_counts.index, y=age_group_counts.values, palette='plasma')
        plt.title('연령대별 공공자전거 이용 분포')
        plt.xlabel('연령대')
        plt.ylabel('이용 횟수')
        plt.xticks(rotation=45)
        plt.tight_layout()

        plot_path = "age_group_distribution_plot.png"
        plt.savefig(plot_path)
        plt.close()
        return plot_path
    else:
        return "'연령대' column not found for plotting."

def plot_top_rental_stations(df, top_n=10):
    if df is None or df.empty:
        return None
    if '대여소' in df.columns and '이용건수' in df.columns:
        station_usage = df.groupby('대여소')['이용건수'].sum().nlargest(top_n).reset_index()

        plt.figure(figsize=(12, 7))
        sns.barplot(x='이용건수', y='대여소', data=station_usage, palette='viridis')
        plt.title(f'이용건수 상위 {top_n}개 대여소')
        plt.xlabel('총 이용 건수')
        plt.ylabel('대여소')
        plt.tight_layout()

        plot_path = "top_rental_stations_plot.png"
        plt.savefig(plot_path)
        plt.close()
        return plot_path
    else:
        return "'대여소' or '이용건수' column not found for plotting."

def plot_trip_duration_distribution(df):
    if df is None or df.empty:
        return None
    if '이용시간(분)' in df.columns:
        plt.figure(figsize=(10, 6))
        sns.histplot(df['이용시간(분)'], bins=50, kde=True)
        plt.title('공공자전거 이용 시간 분포 (분)')
        plt.xlabel('이용 시간 (분)')
        plt.ylabel('빈도수')
        plt.tight_layout()

        plot_path = "trip_duration_distribution_plot.png"
        plt.savefig(plot_path)
        plt.close()
        return plot_path
    else:
        return "'이용시간(분)' column not found for plotting."

# Gradio UI 생성 (여러 탭으로 구성)
with gr.Blocks() as demo:
    gr.Markdown("""# 서울시 공공자전거 이용정보 기본 통계 분석\n\n이 앱은 업로드된 공공자전거 데이터를 병합하고 다양한 통계 정보를 제공

    if 'merged_df' in locals() and not merged_df.empty:
        with gr.Tab("데이터 미리보기"):

```

```

gr.Markdown("### 병합된 데이터프레임의 상위 5행")
gr.HTML(get_dataframe_head(merged_df))
gr.Markdown("### 데이터프레임 정보")
gr.Textbox(value=get_dataframe_info(merged_df), lines=20, label="DataFrame Info")

with gr.Tab("기본 통계"):
    gr.Markdown("### 데이터프레임 기본 기술 통계")
    gr.HTML(get_descriptive_statistics(merged_df))

with gr.Tab("시각화"):
    gr.Markdown("### 일별 이용 횟수")
    gr.Image(plot_daily_usage(merged_df), label="일별 이용 횟수 그래프")
    gr.Markdown("### 이용자 성별 분포")
    gr.Image(plot_gender_distribution(merged_df), label="이용자 성별 분포 그래프")

with gr.Tab("시간대별 이용 현황"):
    gr.Markdown("### 시간대별 공공자전거 이용 현황")
    gr.Image(plot_hourly_usage(merged_df), label="시간대별 이용 현황 그래프")

with gr.Tab("연령대별 이용 분포"):
    gr.Markdown("### 연령대별 공공자전거 이용 분포")
    gr.Image(plot_age_group_distribution(merged_df), label="연령대별 이용 분포 그래프")

with gr.Tab("대여소별 이용 건수"):
    gr.Markdown("### 이용건수 상위 대여소")
    gr.Image(plot_top_rental_stations(merged_df), label="이용건수 상위 대여소 그래프")

with gr.Tab("이용 시간 분포"):
    gr.Markdown("### 공공자전거 이용 시간 분포")
    gr.Image(plot_trip_duration_distribution(merged_df), label="이용 시간 분포 그래프")

else:
    gr.Markdown("### 데이터 로드 오류\n\n데이터프레임을 로드하는 데 문제가 발생했습니다. 위 셀을 실행하여 파일이 올바르게 업로드되었는지 확인하십시오.")

# Gradio 앱 실행
demo.launch(share=True)

```

Matplotlib configured to use NanumBarunGothic.

/tmp/ipykernel_208/1081138001.py:84: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend`

```
sns.barplot(x=gender_counts.index, y=gender_counts.values, palette='viridis')
```

/tmp/ipykernel_208/1081138001.py:108: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend`

```
sns.barplot(x='hour', y='count', data=hourly_usage, palette='magma')
```

/tmp/ipykernel_208/1081138001.py:128: FutureWarning:

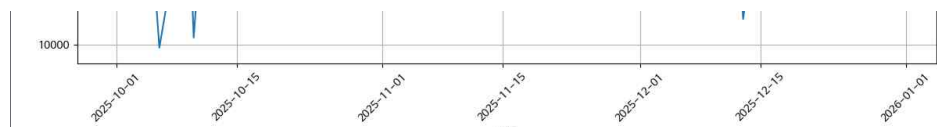
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend`

```
sns.barplot(x=age_group_counts.index, y=age_group_counts.values, palette='plasma')
```

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()

* Running on public URL: <https://ee15a8aa2d5b871661.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working dir



다음 단계: Cloud Run에 배포

```
import requests
import pandas as pd
import gradio as gr

def get_user_by_user_id(user_id):
    if not user_id or user_id < 1 or user_id > 10: # JSONPlaceholder only has users 1-10
        return pd.DataFrame([{"Error": "Please enter a User ID between 1 and 10."}]).to_html(index=False)

    url = f"https://jsonplaceholder.typicode.com/users/{user_id}"
    try:
        response = requests.get(url)
        response.raise_for_status() # Raise an exception for HTTP errors
        user_data = response.json()

        # The response for a single user is a dictionary, convert it to a DataFrame
        # json_normalize can handle nested structures like 'address' and 'company'
        df = pd.json_normalize([user_data]) # Wrap in a list to make it a list of records
        return df.to_html(index=False) # Return as HTML table for Gradio display
    except requests.exceptions.RequestException as e:
        return pd.DataFrame([{"Error": f"API 요청 중 오류 발생: {e}"}]).to_html(index=False)
    except Exception as e:
        return pd.DataFrame([{"Error": f"데이터 처리 중 오류 발생: {e}"}]).to_html(index=False)

# Gradio 인터페이스 생성
iface = gr.Interface(
    fn=get_user_by_user_id,
    inputs=gr.Number(label="Enter User ID (1-10)", precision=0),
    outputs=gr.HTML(label="Filtered User Data"), # Use gr.HTML to display the DataFrame HTML
    title="Filter User Data by User ID",
    description="Enter a User ID to fetch and display the user's data from JSONPlaceholder."
)
```

```
# 인터페이스 실행
iface.launch(share=True)
```

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: <https://22c732d67f359645fd.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working dir

Filter User Data by User ID

Enter a User ID to fetch and display the user's data from JSONPlaceholder.

Enter User ID (1-10)

5

Clear

Submit

id	name	username	email	photo
5	Chelsey Dietrich	Kamren	Lucio_Hettinger@annie.ca	(25x128px)

Flag

Gradio로 제작됨 🍷 · 설정 ⚙️

다음 단계:  Cloud Run에 배포

```
pip install gradio
```

```
Requirement already satisfied: gradio in /usr/local/lib/python3.12/dist-packages (5.50.0)
Requirement already satisfied: aiofiles<25.0,>=22.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (24.1.0)
Requirement already satisfied: anyio<5.0,>=3.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (4.12.1)
Requirement already satisfied: brotli>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (1.2.0)
Requirement already satisfied: fastapi<1.0,>=0.115.2 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.115.1)
Requirement already satisfied: ffmpy in /usr/local/lib/python3.12/dist-packages (from gradio) (1.0.0)
Requirement already satisfied: gradio-client==1.14.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (1.14.0)
Requirement already satisfied: groovy~=0.1 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.1.2)
Requirement already satisfied: httpx<1.0,>=0.24.1 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.28.1)
Requirement already satisfied: huggingface-hub<2.0,>=0.33.5 in /usr/local/lib/python3.12/dist-packages (from gradio) (1.7.1)
Requirement already satisfied: jinja2<4.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.1.6)
Requirement already satisfied: markupsafe<4.0,>=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.0.3)
Requirement already satisfied: numpy<3.0,>=1.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.0.2)
Requirement already satisfied: orjson~=3.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.11.7)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from gradio) (26.0)
Requirement already satisfied: pandas<3.0,>=1.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.2.2)
Requirement already satisfied: pillow<12.0,>=8.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (11.3.0)
Requirement already satisfied: pydantic<=2.12.3,>=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.12.3)
Requirement already satisfied: pydub in /usr/local/lib/python3.12/dist-packages (from gradio) (0.25.1)
Requirement already satisfied: python-multipart>=0.0.18 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.0.22)
Requirement already satisfied: pyyaml<7.0,>=5.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (6.0.3)
Requirement already satisfied: ruff>=0.9.3 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.15.6)
Requirement already satisfied: safehttpx<0.2.0,>=0.1.6 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.1.7)
Requirement already satisfied: semantic-version~=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.10.0)
Requirement already satisfied: starlette<1.0,>=0.40.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.52.1)
Requirement already satisfied: tomlikit<0.14.0,>=0.12.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.13.3)
Requirement already satisfied: typer<1.0,>=0.12 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.24.1)
Requirement already satisfied: typing-extensions~=4.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (4.15.0)
Requirement already satisfied: uvicorn>=0.14.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.42.0)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from gradio-client==1.14.0->gradio) (2025.3.0)
Requirement already satisfied: websockets<16.0,>=13.0 in /usr/local/lib/python3.12/dist-packages (from gradio-client==1.14.0->gradio) (13.1)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5.0,>=3.0->gradio) (3.11)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from fastapi<1.0,>=0.115.2->gradio) (0.12.1)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from fastapi<1.0,>=0.115.2->gradio) (0.0.2)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1.0,>=0.24.1->gradio) (2026.2.25)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1.0,>=0.24.1->gradio) (1.0.9)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1.0,>=0.24.1->gradio) (0.14.0)
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.33.5->gradio) (3.16.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.33.5->gradio) (1.2.1)
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.33.5->gradio) (4.67.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas<3.0,>=1.0->gradio) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas<3.0,>=1.0->gradio) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas<3.0,>=1.0->gradio) (2025.3)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<=2.12.3,>=2.0->gradio) (0.7.0)
```

Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<=2.12.3,>=2.0->gradio) (2.41.4)
 Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer<1.0,>=0.12->gradio) (8.3.1)
 Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer<1.0,>=0.12->gradio) (1.5.4)
 Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer<1.0,>=0.12->gradio) (13.9.4)
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas<3.0,>=1.0->gradio) (1.5.4)
 Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer<1.0,>=0.12->gradio) (3.0.0)
 Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer<1.0,>=0.12->gradio) (2.18.0)
 Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->typer<1.0,>=0.12->gradio) (0.1.2)

```
import gradio as gr

# 1. 텍스트 생성 핵심 로직 (여기에 실제 AI 모델을 연결하면 됩니다)
def generate_text(prompt):
    """
    사용자의 입력을 받아 텍스트를 생성하는 함수입니다.
    ※ 현재는 테스트를 위해 입력값을 메아리처럼 흉내 내는 가짜(Mock) 데이터를 반환합니다.
    ※ 실제 서비스에서는 여기에 ChatGPT API나 오픈소스 LLM 호출 코드를 넣으세요!
    """
    if not prompt:
        return "프롬프트를 입력해 주세요."

    # 가상의 AI 생성 결과
    result = f"입력하신 프롬프트: '{prompt}'\n\n[AI가 생성한 텍스트가 이곳에 출력됩니다. 실제 모델을 연결해 멋진 문장을 만들어보세요!]"
    return result

# 2. 화면 UI 구성 (Gradio Blocks 사용)
with gr.Blocks() as demo:
    gr.Markdown("# 🤖 나만의 텍스트 생성 AI")
    gr.Markdown("원하는 주제를 입력하고 버튼을 누르세요.")

    with gr.Row():
        # 왼쪽 단: 입력부
        with gr.Column():
            input_prompt = gr.Textbox(
                label="프롬프트 (명령어)",
                placeholder="예: 인공지능의 미래에 대해 3줄로 요약해 줘.",
                lines=5
            )
            submit_btn = gr.Button("생성하기", variant="primary")

        # 오른쪽 단: 출력부
        with gr.Column():
            output_result = gr.Textbox(
                label="AI 생성 결과",
                lines=7,
                interactive=False # 사용자가 직접 수정하지 못하게 막음
            )

    # 3. 버튼 클릭 시 동작 설정 (입력 텍스트 -> 함수 -> 출력 텍스트)
    submit_btn.click(
        fn=generate_text,
        inputs=input_prompt,
        outputs=output_result
    )

# 4. 앱 실행
if __name__ == "__main__":
    demo.launch()
```

It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically setting `share=True` (you can turn this off by setting `share=False` in `launch()`).

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()